

HDOC – Experiment 28

1. 2D VLA: Column-wise Input and Row-major Output

Concept

Suppose the user enters:

- Rows = 2
- Columns = 3

❖ Algorithm

1. Start.
2. Declare integers rows, cols, i, j, and k.
3. Read the number of rows.
4. If rows ≤ 0 , display an error message and repeat until a positive value is entered.
5. Read the number of columns.
6. If cols ≤ 0 , display an error message and repeat until a positive value is entered.
7. Declare the variable-length 2D array a[rows][cols].
8. Set j = 0.
9. Repeat while j < cols:
 - Set i = 0.
 - Repeat while i < rows:
 - Read a[i][j].
 - Increment i.

- Increment j.

10. Set k = 0.

11. Repeat while k < rows * cols:

- Calculate row index as k / cols.
- Calculate column index as k % cols.
- Print a[k / cols][k % cols].
- If (k + 1) % cols == 0, print a newline.
- Increment k.

12. Stop.

❖ C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int rows, cols;
```

```
    int i, j, k;
```

```
    /* Read and validate number of rows */
```

```
    do
```

```
    {
```

```
        printf("Enter the number of rows: ");
```

```
        scanf("%d", &rows);
```

```
        if (rows <= 0)
```

```
        {
```

```

        printf("Number of rows must be positive.\n");
    }

} while (rows <= 0);

/* Read and validate number of columns */
do
{
    printf("Enter the number of columns: ");
    scanf("%d", &cols);

    if (cols <= 0)
    {
        printf("Number of columns must be positive.\n");
    }

} while (cols <= 0);

/* Variable-length 2D array */
int a[rows][cols];

printf("\nEnter the array elements column by column:\n");

/* Column-major input */
for (j = 0; j < cols; j++)
{
    for (i = 0; i < rows; i++)
    {

```

```
        printf("a[%d][%d] = ", i, j);
        scanf("%d", &a[i][j]);
    }
}

printf("\nArray elements in row-major order:\n");

/* Row-major output using only one loop */
for (k = 0; k < rows * cols; k++)
{
    printf("%d ", a[k / cols][k % cols]);

    if ((k + 1) % cols == 0)
    {
        printf("\n");
    }
}

return 0;
}
```

❖ Test case table

Test Case	Input	Expected Output	Result
1	Rows=2, Cols=3; column-wise: 1 2 3 4 5 6	1 3 5 / 2 4 6	Pass
2	Rows=1, Cols=3; 10 20 30	10 20 30	Pass
3	Rows=2, Cols=2; 5 6 7 8	5 7 / 6 8	Pass
4	Rows=-2, then 2; Cols=3	Reject -2, accept 2	Pass
5	Rows=2; Cols=0, then 2	Reject 0, accept 2	Pass

2. Copy a 2D VLA into a 1D Array in Row-major Order

Here the program will:

1. Take rows and columns.

2. Create a VLA:

```
int a[rows][cols];
```

3. Read its elements.

4. Create a 1D VLA:

```
int b[rows * cols];
```

5. Copy the 2D array into b in row-major order.

6. Display both arrays.

❖ Algorithm

1. Start.
2. Declare rows, cols, i, j, and k.
3. Read rows and validate that it is positive.
4. Read cols and validate that it is positive.
5. Declare a[rows][cols].
6. Declare b[rows * cols].
7. Set i = 0.
8. For every row:
 - For every column:
 - Read a[i][j].
9. Set k = 0.
10. For every row i:
 - For every column j:
 - Assign b[k] = a[i][j].
 - Increment k.
11. Display the 2D array using nested loops.
12. Display the 1D array using a single loop.
13. Stop.

❖ C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int rows, cols;

int i, j, k;

/* Read and validate rows */

do
{
    printf("Enter the number of rows: ");
    scanf("%d", &rows);

    if (rows <= 0)
    {
        printf("Number of rows must be positive.\n");
    }

} while (rows <= 0);

/* Read and validate columns */

do
{
    printf("Enter the number of columns: ");
    scanf("%d", &cols);

    if (cols <= 0)
    {
        printf("Number of columns must be positive.\n");
    }

} while (cols <= 0);
```

```
/* Variable-length arrays */

int a[rows][cols];

int b[rows * cols];


printf("\nEnter the elements of the 2D array:\n");


/* Read the 2D array */
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        printf("a[%d][%d] = ", i, j);
        scanf("%d", &a[i][j]);
    }
}


/* Copy into 1D array in row-major order */
k = 0;


for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        b[k] = a[i][j];
        k++;
    }
}
```

```
/* Display 2D array */
printf("\n2D Array:\n");

for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        printf("%d ", a[i][j]);

    }

    printf("\n");
}

/* Display 1D array */
printf("\n1D Array in row-major order:\n");

for (k = 0; k < rows * cols; k++)
{
    printf("%d ", b[k]);
}

printf("\n");

return 0;
}
```

❖ Compile, Execute and Evaluate

```
(kali㉿kali)-[~/Desktop/exp28]
$ vi array1.c

(kali㉿kali)-[~/Desktop/exp28]
$ gcc array1.c -o array1

(kali㉿kali)-[~/Desktop/exp28]
$ ./array1
Enter the number of rows: 2
Enter the number of columns: 3

Enter the elements of the 2D array:
a[0][0] = 1
a[0][1] = 2
a[0][2] = 3
a[1][0] = 4
a[1][1] = 5
a[1][2] = 6

2D Array:
1 2 3
4 5 6

1D Array in row-major order:
1 2 3 4 5 6

(kali㉿kali)-[~/Desktop/exp28]
$ ./array1
Enter the number of rows: 1
Enter the number of columns: 3

Enter the elements of the 2D array:
a[0][0] = 10
a[0][1] = 20
a[0][2] = 30

2D Array:
10 20 30
```

```
1D Array in row-major order:  
10 20 30
```

```
(kali㉿kali)-[~/Desktop/exp28]
```

```
$ ./array1
```

```
Enter the number of rows: 2
```

```
Enter the number of columns: 2
```

```
Enter the elements of the 2D array:
```

```
a[0][0] = 5
```

```
a[0][1] = 6
```

```
a[1][0] = 7
```

```
a[1][1] = 8
```

```
2D Array:
```

```
5 6
```

```
7 8
```

```
1D Array in row-major order:
```

```
5 6 7 8
```

```
(kali㉿kali)-[~/Desktop/exp28]
```

```
$ ./array1
```

```
Enter the number of rows: -2
```

```
Number of rows must be positive.
```

```
Enter the number of rows: 2
```

```
Enter the number of columns: 3
```

```
Enter the elements of the 2D array:
```

```
a[0][0] = 5
```

```
a[0][1] = 4
```

```
a[0][2] = 9
```

```
a[1][0] = 7
```

```
a[1][1] = 2
```

```
a[1][2] = 0
```

```
2D Array:
```

```
5 4 9
```

2D Array:

5 4 9
7 2 0

1D Array in row-major order:

5 4 9 7 2 0

(kali@kali)-[~/Desktop/exp28]

\$./array1

Enter the number of rows: 2

Enter the number of columns: 0

Number of columns must be positive.

Enter the number of columns: 2

Enter the elements of the 2D array:

a[0][0] = 8

a[0][1] = 7

a[1][0] = 6

a[1][1] = 5

2D Array:

8 7

6 5

1D Array in row-major order:

8 7 6 5

(kali@kali)-[~/Desktop/exp28]

\$